

1. Setup & Imports

```
import os

import numpy as np

import cv2

import tensorflow as tf

import matplotlib.pyplot as plt

import gradio as gr

from sklearn.metrics import accuracy_score

from tensorflow.keras import layers, models

from tensorflow.keras.applications import ResNet50

from sklearn.model_selection import train_test_split
```

Explanation:

- **os**: For file path handling.
- **numpy (np)**: Numerical operations on arrays.
- **cv2 (OpenCV)**: Image processing (resizing, color conversion).
- **tensorflow (tf)**: Deep learning framework.
- **matplotlib**: Visualization (plotting training curves).
- **gradio (gr)**: Web interface for model deployment.
- **accuracy_score**: Evaluation metric.
- **layers & models**: Keras layers for building neural networks.
- **ResNet50**: Pre-trained CNN for transfer learning.
- **train_test_split**: Splitting data into training/validation sets.

2. Data Loading & Preprocessing

```
IMG_SIZE = 128 # All images resized to 128x128

def load_data(image_dir, mask_dir):

    images, masks = [], []

    image_files = sorted(os.listdir(image_dir))

    mask_files = sorted(os.listdir(mask_dir))

    for img_file, mask_file in zip(image_files, mask_files):

        # Read images and masks
```

```

img = cv2.imread(os.path.join(image_dir, img_file))
mask = cv2.imread(os.path.join(mask_dir, mask_file), cv2.IMREAD_GRAYSCALE)
if img is None or mask is None:
    continue # Skip corrupted files
# Resize and normalize images
img = cv2.resize(img, (IMG_SIZE, IMG_SIZE)) / 255.0 # Normalize to [0,1]
mask = cv2.resize(mask, (IMG_SIZE, IMG_SIZE))
mask = np.expand_dims((mask > 0).astype(np.uint8), axis=-1) # Binary mask (0 or 1)
images.append(img)
masks.append(mask)
return np.array(images), np.array(masks)

```

Explanation:

- **IMG_SIZE:** Standardizes input size for the neural network.
- **load_data():**
 - Reads CT scans (image_dir) and corresponding masks (mask_dir).
 - **Preprocessing Steps:**
 1. **Resize:** Images and masks resized to 128x128.
 2. **Normalization:** Pixel values scaled to [0, 1].
 3. **Mask Binarization:** Converts grayscale masks to binary (0=background, 1=infection).
 - Returns:
 - images: Normalized CT scans (shape=[N, 128, 128, 3]).
 - masks: Binary masks (shape=[N, 128, 128, 1]).

3. Model Architecture (U-Net + ResNet50)

```

def build_unet_resnet_hybrid(input_shape=(128, 128, 3)):
    # Input layer
    inputs = tf.keras.Input(shape=input_shape)
    # Load pre-trained ResNet50 (without top layers)
    base_model = ResNet50(include_top=False, weights="imagenet", input_tensor=inputs)

```

```

base_model.trainable = False # Freeze ResNet weights

# Encoder: Extract feature maps from ResNet

skip_connections = [
    base_model.get_layer("conv1_relu").output, # Early features (high resolution)
    base_model.get_layer("conv2_block3_out").output, # Mid-level features
    base_model.get_layer("conv3_block4_out").output, # Deeper features
    base_model.get_layer("conv4_block6_out").output # High-level features
]

x = base_model.get_layer("conv5_block3_out").output # Bottleneck features

# Decoder (U-Net style upsampling)

for skip in reversed(skip_connections):
    # Upsample and concatenate with skip connection
    x = layers.UpSampling2D((2, 2))(x)
    x = layers.Concatenate()([x, skip])

    # Two convolutional layers
    x = layers.Conv2D(256, 3, activation="relu", padding="same")(x)
    x = layers.Conv2D(256, 3, activation="relu", padding="same")(x)

# Final upsampling and output layer
x = layers.UpSampling2D((2, 2))(x)
x = layers.Conv2D(128, 3, activation="relu", padding="same")(x)
x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)

# Binary segmentation output (sigmoid activation)
outputs = layers.Conv2D(1, 1, activation="sigmoid")(x)

return tf.keras.Model(inputs, outputs)

```

Explanation:

- **Hybrid Architecture:**
 - **Encoder:** ResNet50 (pre-trained on ImageNet) extracts features.
 - **Decoder:** U-Net-style upsampling with skip connections for precise localization.
- **Key Components:**
 - **Skip Connections:** Preserve spatial details from early layers.
 - **Upsampling:** Recovers original resolution step-by-step.

- **Final Layer:** Sigmoid activation outputs probability mask (0 to 1).

4. Training the Model

Load datasets

```
X_train, y_train = load_data("/content/drive/MyDrive/COVID/frames_train/",  
                             "/content/drive/MyDrive/COVID/masks_train/")
```

```
X_test, y_test = load_data("/content/drive/MyDrive/COVID2/frames_test/",  
                            "/content/drive/MyDrive/COVID2/masks_test/")
```

Build and compile model

```
model = build_unet_resnet_hybrid()
```

```
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
```

Train the model

```
history = model.fit(  
    X_train, y_train,  
    epochs=6,  
    batch_size=8,  
    validation_split=0.1,  
    steps_per_epoch=100  
)
```

Save the trained model

```
model.save("/content/drive/MyDrive/COVID_Models/unet_resnet_model.h5")
```

Explanation:

- **Data Loading:**
 - X_train, y_train: Training CT scans and masks.
 - X_test, y_test: Test set for evaluation.
- **Model Compilation:**
 - **Optimizer:** Adam (adaptive learning rate).
 - **Loss:** Binary cross-entropy (suitable for segmentation).
 - **Metric:** Accuracy.
- **Training:**
 - **6 epochs** (passes through the dataset).

- **Batch size = 8** (small due to memory constraints).
- **10% validation split** (monitor overfitting).

5. Evaluation Metrics

```
def dice_coef(y_true, y_pred, smooth=1e-6):
    y_true_f = y_true.flatten()
    y_pred_f = y_pred.flatten()
    intersection = np.sum(y_true_f * y_pred_f)
    return (2.0 * intersection + smooth) / (np.sum(y_true_f) + np.sum(y_pred_f) + smooth)

def evaluate_model(model, X, y):
    preds = model.predict(X)
    preds = (preds > 0.5).astype(np.uint8) # Threshold at 0.5
    acc = accuracy_score(y.flatten(), preds.flatten())
    dice = dice_coef(y, preds)
    return acc, dice

# Load saved model and evaluate

model =
tf.keras.models.load_model("/content/drive/MyDrive/COVID_Models/unet_resnet_model.h5")

test_acc, test_dice = evaluate_model(model, X_test, y_test)

print(f"Test Accuracy: {test_acc*100:.2f}%")
print(f"Dice Coefficient: {test_dice:.4f}")
```

Explanation:

- **Dice Coefficient:**
 - Measures overlap between predicted and ground truth masks.
 - Range: 0 (no overlap) to 1 (perfect match).
- **Accuracy:**
 - Pixel-wise classification accuracy.
- **Evaluation:**
 - Predicts on test set.
 - Converts probabilities to binary mask (>0.5 threshold).

- Computes accuracy and Dice score.

6. Gradio Web Interface

```
def predict_ct(image):
```

```
    # Preprocess input image
```

```
    img = cv2.resize(image, (IMG_SIZE, IMG_SIZE)) / 255.0
```

```
    img_input = np.expand_dims(img, axis=0) # Add batch dimension
```

```
    # Predict infection mask
```

```
    pred_mask = model.predict(img_input)[0]
```

```
    pred_mask_bin = (pred_mask > 0.5).astype(np.uint8) * 255 # Binary mask (0 or 255)
```

```
    # Visualization
```

```
    original = (img * 255).astype(np.uint8) # Original CT scan
```

```
    mask_vis = cv2.merge([pred_mask_bin] * 3) # Convert to 3-channel for display
```

```
    overlay = cv2.addWeighted(original, 0.7, mask_vis, 0.3, 0) # Blend mask with original
```

```
    # Combine into single image
```

```
    combined = np.hstack([original, mask_vis, overlay])
```

```
    return combined
```

```
# Launch Gradio interface
```

```
iface = gr.Interface(
```

```
    fn=predict_ct,
```

```
    inputs=gr.Image(type="numpy", label="Upload CT Scan"),
```

```
    outputs=gr.Image(type="numpy", label="Original | Mask | Overlay"),
```

```
    title="COVID-19 Lung Infection Segmentation",
```

```
    description="""U-Net + ResNet50 model for detecting COVID-19 infections.
```

```
        <br>Left: Original CT | Middle: Predicted Mask | Right: Overlay""",
```

```
    examples=["example_ct1.jpg", "example_ct2.jpg"]
```

```
)
```

```
iface.launch(share=True) # Generates a public URL
```

Explanation:

- **Input:**

- User uploads a CT scan image.
- **Processing:**
 1. Resize and normalize the image.
 2. Predict infection mask using the trained model.
 3. Generate visualizations:
 - **Original:** Input CT scan.
 - **Mask:** Binary prediction (white=infection).
 - **Overlay:** Mask blended with original image.
- **Output:**
 - Side-by-side comparison of results.
- **Deployment:**
 - `share=True` creates a public link for sharing.

Summary of Key Concepts

Component	Purpose
ResNet50 Encoder	Extracts high-level features from CT scans (transfer learning).
U-Net Decoder	Precisely localizes infections using skip connections.
Dice Coefficient	Evaluates segmentation quality (better than accuracy for imbalanced data).
Gradio Interface	Deploys the model as an interactive web app.

```
!pip install tensorflow opencv-python scikit-learn matplotlib gradio
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.11/dist-packages (2.18.0)
```

```
Requirement already satisfied: opencv-python in /usr/local/lib/python3.11/dist-packages (4.11.0.86)
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.11/dist-packages (1.6.1)
```

```
Requirement already satisfied: matplotlib in /usr/local/lib/python3.11/dist-packages (3.10.0)
```

```
Collecting gradio
```

```
Downloading gradio-5.28.0-py3-none-any.whl.metadata (16 kB)
```

Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.4.0)

Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.6.3)

Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (25.2.10)

Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.6.0)

Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.2.0)

Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (18.1.1)

Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.4.0)

Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from tensorflow) (24.2)

Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.21.3,!4.21.4,!4.21.5,<6.0.0dev,>=3.20.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (5.29.4)

Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.32.3)

Requirement already satisfied: setuptools in /usr/local/lib/python3.11/dist-packages (from tensorflow) (75.2.0)

Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.17.0)

Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.0.1)

Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (4.13.2)

Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.17.2)

Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (1.71.0)

Requirement already satisfied: tensorboard<2.19,>=2.18 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.18.0)

Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.8.0)

Requirement already satisfied: numpy<2.1.0,>=1.26.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (2.0.2)

Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (3.13.0)

Requirement already satisfied: ml-dtypes<0.5.0,>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.4.1)

Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in /usr/local/lib/python3.11/dist-packages (from tensorflow) (0.37.1)

Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.15.2)

Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (1.4.2)

Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.11/dist-packages (from scikit-learn) (3.6.0)

Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.3.2)

Requirement already satisfied: cyclor>=0.10 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (4.57.0)

Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (1.4.8)

Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (11.2.1)

Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (3.2.3)

Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.11/dist-packages (from matplotlib) (2.9.0.post0)

Collecting aiofiles<25.0,>=22.0 (from gradio)

Downloading aiofiles-24.1.0-py3-none-any.whl.metadata (10 kB)

Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (4.9.0)

Collecting fastapi<1.0,>=0.115.2 (from gradio)

Downloading fastapi-0.115.12-py3-none-any.whl.metadata (27 kB)

Collecting ffmpeg (from gradio)

Downloading ffmpeg-0.5.0-py3-none-any.whl.metadata (3.0 kB)

Collecting gradio-client==1.10.0 (from gradio)

Downloading gradio_client-1.10.0-py3-none-any.whl.metadata (7.1 kB)

Collecting groovy~=0.1 (from gradio)

Downloading groovy-0.1.2-py3-none-any.whl.metadata (6.1 kB)

Requirement already satisfied: httpx>=0.24.1 in /usr/local/lib/python3.11/dist-packages (from gradio) (0.28.1)

Requirement already satisfied: huggingface-hub>=0.28.1 in /usr/local/lib/python3.11/dist-packages (from gradio) (0.30.2)

Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (3.1.6)

Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (3.0.2)

Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (3.10.16)

Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (2.2.2)

Requirement already satisfied: pydantic<2.12,>=2.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (2.11.3)

Collecting pydub (from gradio)

Downloading pydub-0.25.1-py2.py3-none-any.whl.metadata (1.4 kB)

Collecting python-multipart>=0.0.18 (from gradio)

Downloading python_multipart-0.0.20-py3-none-any.whl.metadata (1.8 kB)

Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.11/dist-packages (from gradio) (6.0.2)

Collecting ruff>=0.9.3 (from gradio)

Downloading ruff-0.11.8-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (25 kB)

Collecting safehttpx<0.2.0,>=0.1.6 (from gradio)

Downloading safehttpx-0.1.6-py3-none-any.whl.metadata (4.2 kB)

Collecting semantic-version~=2.0 (from gradio)

Downloading semantic_version-2.10.0-py2.py3-none-any.whl.metadata (9.7 kB)

Collecting starlette<1.0,>=0.40.0 (from gradio)

Downloading starlette-0.46.2-py3-none-any.whl.metadata (6.2 kB)

Collecting tomllkit<0.14.0,>=0.12.0 (from gradio)

Downloading tomli-0.13.2-py3-none-any.whl.metadata (2.7 kB)

Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.11/dist-packages (from gradio) (0.15.2)

Collecting uvicorn>=0.14.0 (from gradio)

Downloading uvicorn-0.34.2-py3-none-any.whl.metadata (6.5 kB)

Requirement already satisfied: fsspec in /usr/local/lib/python3.11/dist-packages (from gradio-client==1.10.0->gradio) (2025.3.2)

Requirement already satisfied: websockets<16.0,>=10.0 in /usr/local/lib/python3.11/dist-packages (from gradio-client==1.10.0->gradio) (15.0.1)

Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.11/dist-packages (from anyio<5.0,>=3.0->gradio) (3.10)

Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python3.11/dist-packages (from anyio<5.0,>=3.0->gradio) (1.3.1)

Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.11/dist-packages (from astunparse>=1.6.0->tensorflow) (0.45.1)

Requirement already satisfied: certifi in /usr/local/lib/python3.11/dist-packages (from httpx>=0.24.1->gradio) (2025.1.31)

Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.11/dist-packages (from httpx>=0.24.1->gradio) (1.0.9)

Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.11/dist-packages (from httpcore==1.*->httpx>=0.24.1->gradio) (0.16.0)

Requirement already satisfied: filelock in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.28.1->gradio) (3.18.0)

Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python3.11/dist-packages (from huggingface-hub>=0.28.1->gradio) (4.67.1)

Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)

Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow) (0.0.9)

Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras>=3.5.0->tensorflow) (0.15.0)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/dist-packages (from pandas<3.0,>=1.0->gradio) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/dist-packages (from pandas<3.0,>=1.0->gradio) (2025.2)

Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.11/dist-packages (from pydantic<2.12,>=2.0->gradio) (0.7.0)

Requirement already satisfied: pydantic-core==2.33.1 in /usr/local/lib/python3.11/dist-packages (from pydantic<2.12,>=2.0->gradio) (2.33.1)

Requirement already satisfied: typing-inspection>=0.4.0 in /usr/local/lib/python3.11/dist-packages (from pydantic<2.12,>=2.0->gradio) (0.4.0)

Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (3.4.1)

Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests<3,>=2.21.0->tensorflow) (2.4.0)

Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow) (3.8)

Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow) (0.7.2)

Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.11/dist-packages (from tensorboard<2.19,>=2.18->tensorflow) (3.1.3)

Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python3.11/dist-packages (from typer<1.0,>=0.12->gradio) (8.1.8)

Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.11/dist-packages (from typer<1.0,>=0.12->gradio) (1.5.4)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow) (3.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras>=3.5.0->tensorflow) (2.19.1)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from markdown-it-py>=2.2.0->rich->keras>=3.5.0->tensorflow) (0.1.2)

Downloading gradio-5.28.0-py3-none-any.whl (54.1 MB)

54.1/54.1 MB 8.5 MB/s eta 0:00:00

Downloading gradio_client-1.10.0-py3-none-any.whl (322 kB)

322.9/322.9 kB 8.8 MB/s eta 0:00:00

Downloading aiofiles-24.1.0-py3-none-any.whl (15 kB)

Downloading fastapi-0.115.12-py3-none-any.whl (95 kB)

95.2/95.2 kB 4.6 MB/s eta 0:00:00

Downloading groovy-0.1.2-py3-none-any.whl (14 kB)

Downloading python_multipart-0.0.20-py3-none-any.whl (24 kB)

Downloading ruff-0.11.8-py3-none-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (11.5 MB)

11.5/11.5 MB 30.3 MB/s eta 0:00:00

Downloading safehttpx-0.1.6-py3-none-any.whl (8.7 kB)

Downloading semantic_version-2.10.0-py2.py3-none-any.whl (15 kB)

Downloading starlette-0.46.2-py3-none-any.whl (72 kB)

72.0/72.0 kB 2.8 MB/s eta 0:00:00

Downloading tomlkit-0.13.2-py3-none-any.whl (37 kB)

Downloading uvicorn-0.34.2-py3-none-any.whl (62 kB)

62.5/62.5 kB 2.5 MB/s eta 0:00:00

Downloading ffmpeg-0.5.0-py3-none-any.whl (6.0 kB)

Downloading pydub-0.25.1-py2.py3-none-any.whl (32 kB)

Installing collected packages: pydub, uvicorn, tomlkit, semantic-version, ruff, python-multipart, groovy, ffmpeg, aiofiles, starlette, safehttpx, gradio-client, fastapi, gradio

Successfully installed aiofiles-24.1.0 fastapi-0.115.12 ffmpeg-0.5.0 gradio-5.28.0 gradio-client-1.10.0 groovy-0.1.2 pydub-0.25.1 python-multipart-0.0.20 ruff-0.11.8 safehttpx-0.1.6 semantic-version-2.10.0 starlette-0.46.2 tomlkit-0.13.2 uvicorn-0.34.2# Import required libraries

```
from google.colab import drive
```

```
drive.mount('/content/drive')
```

Mounted at /content/drive

```
from sklearn.model_selection import train_test_split
```

```
import os
```

```
import numpy as np
```

```
import cv2
```

```
import tensorflow as tf
```

```
import matplotlib.pyplot as plt
```

```
import gradio as gr

from sklearn.metrics import accuracy_score

from tensorflow.keras import layers, models

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras.applications import ResNet50, DenseNet121, MobileNetV2, InceptionV3,
InceptionResNetV2
```

```
# Constants
```

```
IMG_SIZE = 128
```

```
IMG_SIZE = 128
```

```
def load_data(image_dir, mask_dir):
```

```
    images, masks = [], []
```

```
    image_files = sorted(os.listdir(image_dir))
```

```
    mask_files = sorted(os.listdir(mask_dir))
```

```
    for img_file, mask_file in zip(image_files, mask_files):
```

```
        img_path = os.path.join(image_dir, img_file)
```

```
        mask_path = os.path.join(mask_dir, mask_file)
```

```
        img = cv2.imread(img_path)
```

```
        mask = cv2.imread(mask_path, cv2.IMREAD_GRAYSCALE)
```

```
        if img is None or mask is None:
```

```
            continue
```

```
        img = cv2.resize(img, (IMG_SIZE, IMG_SIZE)) / 255.0
```

```
        mask = cv2.resize(mask, (IMG_SIZE, IMG_SIZE))
```

```
        mask = np.expand_dims((mask > 0).astype(np.uint8), axis=-1)
```

```
    images.append(img)
```

```

        masks.append(mask)

    return np.array(images), np.array(masks)

# Load Dataset 1 (for training)
X_train, y_train = load_data('/content/drive/MyDrive/COVID/frames_train/',
                              '/content/drive/MyDrive/COVID/masks_train/')

# Load Dataset 2 (for testing)
X_test, y_test = load_data('/content/drive/MyDrive/COVID2/frames_test/',
                             '/content/drive/MyDrive/COVID2/masks_test/')

    if img is None or mask is None:
        continue

def build_unet_resnet_hybrid(input_shape=(IMG_SIZE, IMG_SIZE, 3)):
    inputs = tf.keras.Input(shape=input_shape)

    base_model = tf.keras.applications.ResNet50(include_top=False, weights='imagenet',
        input_tensor=inputs)

    base_model.trainable = False

    skips = [
        base_model.get_layer("conv1_relu").output,
        base_model.get_layer("conv2_block3_out").output,
        base_model.get_layer("conv3_block4_out").output,
        base_model.get_layer("conv4_block6_out").output
    ]

    x = base_model.get_layer("conv5_block3_out").output

    for skip in reversed(skips):
        x = tf.keras.layers.UpSampling2D((2, 2))(x)

```

```

x = tf.keras.layers.Concatenate()([x, skip])
x = tf.keras.layers.Conv2D(256, 3, activation='relu', padding='same')(x)
x = tf.keras.layers.Conv2D(256, 3, activation='relu', padding='same')(x)

x = tf.keras.layers.UpSampling2D((2, 2))(x)
x = tf.keras.layers.Conv2D(128, 3, activation='relu', padding='same')(x)
x = tf.keras.layers.Conv2D(64, 3, activation='relu', padding='same')(x)

outputs = tf.keras.layers.Conv2D(1, 1, activation='sigmoid')(x)
return tf.keras.Model(inputs, outputs)

# Train the model
history = model.fit(
    X_train,
    y_train,
    epochs=6,
    batch_size=8,
    validation_split=0.1,
    steps_per_epoch=100
)

# Save the model
model.save('/content/drive/MyDrive/COVID_Models/unet_resnet_model.h5')
model = build_unet_resnet_hybrid()
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.fit(X_train, y_train, epochs=6, batch_size=8, validation_split=0.1, steps_per_epoch=100)
model.save('/content/drive/MyDrive/COVID_Models/unet_resnet_model.h5')
Epoch 1/6

```

100/100 ————— **64s** 317ms/step - accuracy: 0.9360 - loss: 0.2295 - val_accuracy: 0.9656 - val_loss: 0.1920

Epoch 2/6

100/100 ————— **16s** 157ms/step - accuracy: 0.9843 - loss: 0.0645 - val_accuracy: 0.9656 - val_loss: 0.1664

Epoch 3/6

100/100 ————— **16s** 160ms/step - accuracy: 0.9833 - loss: 0.0596 - val_accuracy: 0.9656 - val_loss: 0.3690

Epoch 4/6

7/100 ————— **13s** 144ms/step - accuracy: 0.9864 - loss: 0.0536

/usr/local/lib/python3.11/dist-packages/keras/src/trainers/epoch_iterator.py:107: UserWarning: Your input ran out of data; interrupting training. Make sure that your dataset or generator can generate at least `steps\_per\_epoch \* epochs` batches. You may need to use the `.repeat()` function when building your dataset.

```
self._interrupted_warning()
```

100/100 ————— **4s** 35ms/step - accuracy: 0.9885 - loss: 0.0474 - val_accuracy: 0.9656 - val_loss: 0.1759

Epoch 5/6

100/100 ————— **16s** 163ms/step - accuracy: 0.9843 - loss: 0.0579 - val_accuracy: 0.9656 - val_loss: 0.1692

Epoch 6/6

100/100 ————— **16s** 162ms/step - accuracy: 0.9852 - loss: 0.0546 - val_accuracy: 0.9656 - val_loss: 0.1461

WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save\_model(model)`. This file format is considered legacy. We recommend using instead the native Keras format, e.g. `model.save('my\_model.keras')` or `keras.saving.save\_model(model, 'my\_model.keras')`.

```
def evaluate_model(model, X, y):
```

```
    preds = model.predict(X)
```

```
    preds = (preds > 0.5).astype(np.uint8)
```

```
    acc = accuracy_score(y.flatten(), preds.flatten())
```

```
    return acc
```

```
unet_resnet_model =
```

```
tf.keras.models.load_model('/content/drive/MyDrive/COVID_Models/unet_resnet_model.h5')
```

```
test_acc = evaluate_model(unet_resnet_model, X_test, y_test)
```

```
print(f"✅ Test Accuracy on Dataset 2: {test_acc*100:.2f}%")
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built.
`model.compile\_metrics` will be empty until you train or evaluate the model.

86/86 ————— **48s** 334ms/step

✅ Test Accuracy on Dataset 2: 98.34%

```
def dice_coef(y_true, y_pred, smooth=1e-6):
```

```
    y_true_f = y_true.flatten()
```

```
    y_pred_f = y_pred.flatten()
```

```
    intersection = np.sum(y_true_f * y_pred_f)
```

```
    return (2. * intersection + smooth) / (np.sum(y_true_f) + np.sum(y_pred_f) + smooth)
```

```
preds = (unet_resnet_model.predict(X_test) > 0.5).astype(np.uint8)
```

```
dice = dice_coef(y_test, preds)
```

```
print(f"🧊 Dice Coefficient on Dataset 2: {dice:.4f}")
```

86/86 ————— **17s** 192ms/step

🧊 Dice Coefficient on Dataset 2: 0.0000

```
import gradio as gr
```

```
import cv2
```

```
# Load trained model
```

```
model =
```

```
tf.keras.models.load_model('/content/drive/MyDrive/COVID_Models/unet_resnet_model.h5')
```

```
# Prediction function
```

```
def predict_ct(image):
```

```
    # Resize to match model input
```

```
    image_resized = cv2.resize(image, (IMG_SIZE, IMG_SIZE)) / 255.0
```

```
    image_input = np.expand_dims(image_resized, axis=0) # Add batch dimension
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built.
`model.compile\_metrics` will be empty until you train or evaluate the model.

```

def predict_ct(image):
    image_resized = cv2.resize(image, (IMG_SIZE, IMG_SIZE)) / 255.0
    image_input = np.expand_dims(image_resized, axis=0)

    # Predict the mask
    pred_mask = model.predict(image_input)[0]
    pred_mask = (pred_mask > 0.5).astype(np.uint8) * 255

    # Create overlay
    overlay = image_resized.copy()
    overlay_mask = pred_mask.squeeze().astype(np.uint8)
    overlay_mask_colored = cv2.merge([overlay_mask]*3)
    overlay = (0.6 * overlay * 255 + 0.4 * overlay_mask_colored).astype(np.uint8)
    return overlay

```

```

def predict_ct(image):
    # Resize and normalize
    image_resized = cv2.resize(image, (IMG_SIZE, IMG_SIZE)) / 255.0
    image_input = np.expand_dims(image_resized, axis=0)

    # Predict the mask
    pred_mask = model.predict(image_input)[0]
    pred_mask_bin = (pred_mask > 0.5).astype(np.uint8) * 255
    pred_mask_bin = pred_mask_bin.squeeze()

    # Convert grayscale mask to 3 channels
    pred_mask_colored = cv2.merge([pred_mask_bin] * 3)

    # Convert original to display range
    image_disp = (image_resized * 255).astype(np.uint8)

```

```

# Overlay mask on original image
overlay = cv2.addWeighted(image_disp, 0.7, pred_mask_colored, 0.3, 0)

# Stack all three: original | mask | overlay
combined = np.hstack([image_disp, pred_mask_colored, overlay])
return combined

gr.Interface(
    fn=predict_ct,
    inputs=gr.Image(type="numpy", label="Upload CT Image"),
    outputs=gr.Image(type="numpy", label="Original | Mask | Overlay"),
    title="COVID-19 CT Scan Segmentation",
    description="Left: Original CT, Middle: Infection Mask, Right: Overlay"
).launch()

```

It looks like you are running Gradio on a hosted Jupyter notebook. For the Gradio app to work, sharing must be enabled. Automatically setting `share=True` (you can turn this off by setting `share=False` in `launch()` explicitly).

Colab notebook detected. To show errors in colab notebook, set debug=True in launch()

* Running on public URL: <https://b9834ba947a84e1a34.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the working directory to deploy to Hugging Face Spaces

<https://huggingface.co/spaces>



Clear

Submit

☒ Infection Mask Overlay



Clear

Submit

☒ Infection Mask Overlay

